

Comparative Performance Analysis of MILP Solvers for Cryptanalysis

Halil İbrahim Kaplan¹[0009–0006–0120–6512]

Cryptanalysis Laboratory, TÜBİTAK BİLGEM, Kocaeli, Turkey
`halil.kaplan@tubitak.gov.tr`

Abstract. This paper provides a performance comparison of five MILP solvers applied to related-key differential cryptanalysis of ITUbee [8]. We evaluate three open-source solvers (GLPK, HiGHS, SCIP) and two commercial solvers (Gurobi, CPLEX) using MILP models for 8, 10, and 12-round attacks.

As rounds increase, the number of equations and constraints grows exponentially. Experiments used an 11th Gen Intel Core i7-1165G7 processor with 32 GB of RAM. Commercial solvers (Gurobi and CPLEX) significantly outperform open-source alternatives, achieving up to $94\times$ speedup compared to GLPK for the 12-round model. This work provides guidance for selecting solvers for MILP-based cryptanalysis.

Keywords: Cryptanalysis · MILP · Differential Cryptanalysis · Block Ciphers

1 Introduction

Lightweight block ciphers are critical for resource-constrained environments such as embedded systems, IoT devices, and sensor networks. ITUbee is a software-oriented lightweight block cipher specifically designed for resource-constrained devices with limited computational power and memory [8]. The cipher features a Feistel structure consisting of 20 rounds, operating on an 80-bit block size with an 80-bit key. The encryption process incorporates a key whitening layer at both the beginning and end of the 20-round structure, employing substitution (F function) and linear transformation (L function) to provide security against various attacks.

Cryptanalytic attacks on block ciphers can be categorized into several fundamental types. Differential cryptanalysis, first introduced by Biham and Shamir [2], exploits statistical properties of plaintext-ciphertext pairs to recover keys. Related-key differential cryptanalysis extends this paradigm by considering differences in both the plaintext and the encryption key, allowing attackers to exploit structural relationships between keys. This attack model is particularly relevant for cipher evaluation as it represents a more powerful threat than standard differential cryptanalysis.

In recent years, Mixed Integer Linear Programming (MILP) has emerged as a systematic and automated approach to finding optimal differential and

related-key differential characteristics, significantly advancing the field of cryptanalysis [9]. The advantage of MILP-based cryptanalysis lies in its automation: researchers can construct objective functions and constraint systems that model the cipher’s operations, and solvers can systematically search for optimal (minimum-weight) differential paths with proven guarantees of optimality.

However, the computational complexity of MILP-based related-key differential cryptanalysis depends critically on solver performance. Different MILP solvers employ different algorithms, heuristics, and optimization strategies, leading to substantial variations in solution time. This paper addresses a practical question of significant importance to cryptanalysts: which MILP solver should be selected for related-key differential cryptanalysis, and what performance improvements can be expected with different solver choices?

Organization of the paper. The rest of the paper is organized as follows. Section 2 ??

2 Our MILP Model

In this study, we adopt the related-key differential cryptanalysis model introduced in [7]. To facilitate a comprehensive comparison, 8-, 10-, and 12-round instances of the model are employed to evaluate the performance of various solvers. For the sake of completeness, a brief overview of related-key differential cryptanalysis and the ITUbee cipher is also provided; for a more detailed exposition, the reader is referred to [7].

2.1 ITUbee

ITUbee [8] is a lightweight software-oriented block cipher specifically designed for resource-constrained devices which include a microcontroller and have a limited battery power such as sensor nodes in wireless sensor networks.

The cipher has Feistel structure which consists of 20 rounds. It has 80 bits block size and 80 bits key size. The round structure of the cipher is given in Figure 1. Cipher also has a key whitening layer in the beginning and end of the 20 rounds. The encryption process of the cipher is given in Algorithm 1.

L Function The L function which has branch number 4 is computed as,
 $L(a \parallel b \parallel c \parallel d \parallel e) = (e \oplus a \oplus b) \parallel (a \oplus b \oplus c) \parallel (b \oplus c \oplus d) \parallel (c \oplus d \oplus e) \parallel (d \oplus e \oplus a)$

F Function The F function is computed as,

$$F(X) = S(L(S(X)))$$

S is the S-box used in AES [11]. S is the only non-linear structure in the ITUbee algorithm. 40 bit input of S is splitted into 8 bits then enters the s-box.

$$S(a \parallel b \parallel c \parallel d \parallel e) = s[a] \parallel s[b] \parallel s[c] \parallel s[d] \parallel s[e]$$

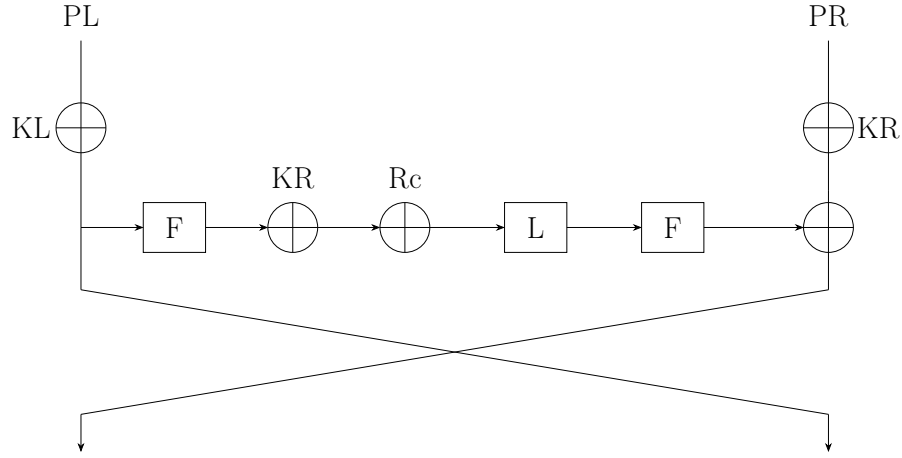


Fig. 1. ITUbee round structure

Algorithm 1 Pseudo code for ITUbee encryption

```

1:  $X_1 \leftarrow P_L \oplus K_L$ 
2:  $X_0 \leftarrow P_R \oplus K_R$ 
3: for  $i = 1$  to 20 do
4:   if  $i \in \{1, 3, \dots, 19\}$  then
5:      $RK \leftarrow K_R$ 
6:   else
7:      $RK \leftarrow K_L$ 
8:   end if
9:    $X_{i+1} \leftarrow X_{i-1} \oplus F(L(RK \oplus RC_i \oplus F(X_i)))$ 
10:  Round constant is added to the rightmost 16 bits
11: end for
12:  $C_L \leftarrow X_{20} \oplus K_R$ 
13:  $C_R \leftarrow X_{21} \oplus K_L$ 

```

Key Schedule ITUbee has no key schedule. The algorithm splits the 80 bit key into two 40 bit parts K_L and K_R and uses the right part of the key in odd rounds and the left part of the keys in even rounds.

2.2 Related-key Differential Cryptanalysis

For related key model, unlike the differential model, we must also take into account the keys. We also define binary variables for the keys and add constraints for activity of at least 1 key bytes. (Otherwise model will never give active bytes for key and it will be same as differential cryptanalysis) In this model we have more middle values than differential and linear model because we also need to chance the variable after the key xor in the middle. Sketch of the one round MILP model of the linear cryptanalysis is given in Figure 2.

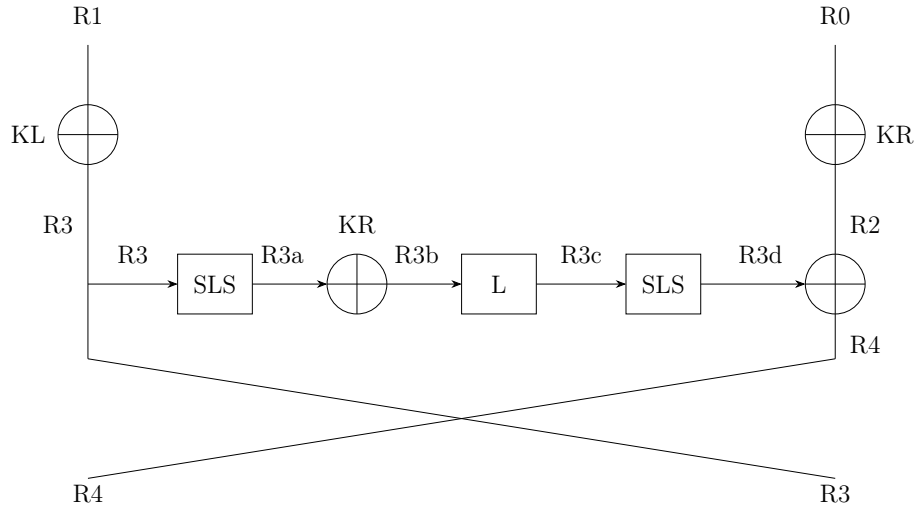


Fig. 2. ITUbee MILP Related-Key Differential Cryptanalysis Sketch

We found that 8 round of the algorithm has minimum 16 active s-box. ITUbee algorithm uses AES's [3] s-box. AES s-box have best input-output difference 2^{-6} . Therefore best probability for differential characteristic will be $(2^{-6})^{16} = 2^{-96}$ which is not usable for related key differential attack. Authors of the ITUbee algorithm claims in [8] that algorithm is secure against related key differential cryptanalysis after 10 round. In our model we found that 8 round is enough for security against related key differential cryptanalysis. Best 8 round characteristic found with the model can be seen in Table 1

3 Solver Specifications

This section provides detailed descriptions of the five MILP solvers evaluated in our comparative study.

3.1 GLPK (GNU Linear Programming Kit)

GLPK is a free and open-source linear programming solver developed as part of the GNU project [10]. Version 4.65 was used in our experiments. GLPK implements the revised simplex method for solving linear programming problems and employs branch-and-cut algorithms for handling integer constraints. While GLPK provides reliable and deterministic solutions, it lacks the advanced heuristics and parallel processing capabilities of commercial solvers, making it relatively slower for complex MILP instances. GLPK is widely used in academic research due to its accessibility and reasonable performance on small to medium-sized problems.

3.2 HiGHS (High-performance Interior-point Solver)

HiGHS (version 0.9) is a modern open-source linear and mixed-integer programming solver developed by the University of Edinburgh [4]. It implements advanced algorithms including the simplex method, interior-point method, and active-set method for LP problems, combined with parallel branch-and-bound techniques for MILP. HiGHS was designed with computational efficiency in mind, featuring parallel processing capabilities and modern data structures. Despite being open-source, HiGHS achieves significantly better performance than GLPK on many problem instances, particularly benefiting from multi-core processors through parallelization.

3.3 SCIP (Solving Constraint Integer Programs)

SCIP (version 7.0) is a powerful open-source constraint integer programming framework developed by the Zuse Institute Berlin [1]. Beyond basic MILP, SCIP supports advanced problem classes including mixed-integer nonlinear programming and quadratically constrained problems. SCIP incorporates sophisticated algorithmic techniques including cutting planes, primal heuristics, and symmetry breaking. Among open-source solvers, SCIP consistently demonstrates superior performance, often approaching the solution quality of commercial solvers for many problem classes. Its flexibility and strength make it a preferred choice for academic researchers with demanding computational requirements.

3.4 Gurobi Optimizer

Gurobi (version 10.0) is a leading commercial MILP solver developed by Gurobi Optimization [5]. It employs state-of-the-art algorithms including barrier methods (interior-point), parallel branch-and-cut, and advanced presolve techniques. Gurobi features sophisticated primal and dual heuristics for finding high-quality solutions quickly, combined with proven-effective cutting plane strategies. The solver automatically distributes computation across multiple processor cores, with dynamic load balancing. Gurobi's extensive tuning and parameter options allow fine-grained control over the solving process. It is widely deployed in industry and has become a benchmark standard for commercial solver performance.

3.5 CPLEX (IBM CPLEX Optimization Studio)

CPLEX (version 12.10) is IBM's flagship commercial optimization solver with decades of refinement [6]. It implements multiple algorithms including primal and dual simplex, barrier methods with crossover, and advanced branch-and-cut with dynamic strategy selection. CPLEX features parallel algorithms that can leverage multiple processor cores effectively. The solver includes sophisticated preprocessing, probing techniques, and performance tuning capabilities. CPLEX has established itself as the de facto standard in enterprise optimization applications and represents the highest maturity level in commercial MILP solving.

3.6 Experimental Configuration

All solvers were executed with default parameters to ensure fair and reproducible comparison. This approach reflects typical usage patterns where practitioners rely on default settings without extensive parameter tuning. Each solver was allowed unlimited computation time (no time limits were imposed) to find optimal solutions. Experiments were conducted on an 11th Gen Intel Core i7-1165G7 processor operating at 2.80 GHz with 32 GB of system RAM, running Ubuntu Linux. This ensures all solvers had equal access to computational resources, with no artificial constraints or advantages.

4 Results and Analysis

Comprehensive benchmarking was conducted on MILP models for 8, 10, and 12-round related-key differential attacks on ITUbee. The results demonstrate clear performance distinctions between solver categories, with significant implications for practical related-key differential cryptanalysis.

4.1 8-Round Results

Our 8-round model consist of 687 variables and 2147 constraints. Our results show that GLPK, the baseline open-source solver, required 257 seconds to find a solution. HiGHS improved on this with a time of 231 seconds, while SCIP further reduced the time to 86 seconds. The commercial solvers Gurobi and CPLEX achieved significantly faster times of 37 and 16 seconds, respectively, representing speedups of $6.9\times$ and $16.0\times$ compared to GLPK (see Table 2).

Table 2. Solution times for the 8-round model

Solver	Time (s)	Speedup
GLPK	257	$1.0\times$
HiGHS	231	$1.1\times$
SCIP	86	$2.9\times$
Gurobi	37	$6.9\times$
CPLEX	16	$16.0\times$

4.2 10-Round Results

The 10-round model consists of 945 variables and 3182 constraints. In this harder instance, GLPK required 1610 seconds to solve the problem. HiGHS improved substantially, finishing in 321 seconds, and SCIP further reduced the time to 202 seconds. The commercial solvers again delivered the best performance, with Gurobi solving the model in 34 seconds and CPLEX in 40 seconds, representing speedups of $47.4\times$ and $40.3\times$ versus GLPK, respectively (see Table 3).

Table 3. Solution times for the 10-round model

Solver	Time (s)	Speedup
GLPK	1610	1.0×
HiGHS	321	5.0×
SCIP	202	7.9×
Gurobi	34	47.4×
CPLEX	40	40.3×

4.3 12-Round Results

The 12-round model is the most challenging benchmark, with 1203 variables and 4217 constraints. GLPK required 4543 seconds to solve this instance, while HiGHS completed in 1780 seconds. SCIP again offered a strong open-source alternative at 376 seconds. Gurobi and CPLEX achieved the fastest times of 48 and 53 seconds, corresponding to speedups of 94.6× and 85.7× relative to GLPK (Table 4).

Table 4. Solution times for the 12-round model

Solver	Time (s)	Speedup
GLPK	4543	1.0×
HiGHS	1780	2.5×
SCIP	376	12.0×
Gurobi	48	94.6×
CPLEX	53	85.7×

4.4 Discussion

The experimental results reveal several important trends:

1. **Commercial vs. Open-Source Gap:** Commercial solvers (Gurobi and CPLEX) consistently outperform open-source alternatives, with speedups increasing dramatically for larger problem sizes. For the 12-round model, Gurobi achieves a 94.6× speedup over GLPK.
2. **Problem Scaling:** As the problem complexity increases (8 to 12 rounds), the performance gap between solvers widens. This suggests that more sophisticated optimization algorithms in commercial solvers are particularly effective on harder instances.
3. **Open-Source Alternatives:** Among open-source solvers, SCIP consistently outperforms both GLPK and HiGHS, achieving 2.9× speedup on 8-rounds and 12× speedup on 12-rounds compared to GLPK.
4. **Commercial Solver Parity:** Gurobi and CPLEX show comparable performance, with Gurobi slightly faster on 10 and 12-round models, while CPLEX excels on the 8-round problem.

The timing analysis demonstrates that solver choice is critical for practical cryptanalysis. For resource-constrained deployments, SCIP provides a reasonable alternative to commercial solvers. However, for optimal performance, commercial solutions are strongly recommended.

5 Conclusion

This paper provides a systematic empirical comparison of five state-of-the-art MILP solvers in the context of cryptanalytic applications, specifically related-key differential attacks on the ITUbee lightweight block cipher. Our findings clearly demonstrate that solver selection significantly impacts the practical feasibility of MILP-based related-key differential cryptanalysis.

Key conclusions include:

1. Commercial MILP solvers (Gurobi and CPLEX) are substantially more efficient than open-source alternatives, justifying their use in computationally demanding related-key differential cryptanalytic applications.
2. For organizations without access to commercial solvers, SCIP represents a robust open-source option, providing reasonable performance for moderate problem sizes in related-key differential attacks.
3. The performance advantage of commercial solvers increases with problem complexity, suggesting that their advanced algorithms are particularly effective for hard instances of related-key differential attacks.
4. Cryptanalysts should carefully evaluate solver options based on their specific constraints (computational budget, licensing availability, problem size) when conducting related-key differential cryptanalysis.

References

1. Berlin, Z.I.: Scip: Solving constraint integer programs. <https://www.scipopt.org/> (2023)
2. Biham, E., Shamir, A.: Differential cryptanalysis of des-like cryptosystems. In: Workshop on the Theory and Application of Cryptographic Techniques. pp. 2–21. Springer (1990)
3. Dworkin, M.J.: The advanced encryption standard (aes) and its use in cryptographic standards. NIST Special Publication **800**, 38 (2001)
4. of Edinburgh, U.: Highs: High-performance interior-point solver. <https://github.com/ERGO-Code/HiGHS> (2023)
5. Gurobi Optimization, L.: Gurobi optimizer. <https://www.gurobi.com/> (2024)
6. IBM: Ibm cplex optimization studio. <https://www.ibm.com/products/ilog-cplex-optimization-studio> (2024)
7. İbrahim Kaplan, H.: Usage of mixed integer linear programming in cryptanalysis of block ciphers. Cryptology ePrint Archive, Paper 2026/224 (2026), <https://eprint.iacr.org/2026/224>
8. Karakoç, F., Demirci, H., Harmancı, A.E.: Itubee: a software oriented lightweight block cipher. In: Lightweight Cryptography for Security and Privacy: Second International Workshop, LightSec 2013, Gebze, Turkey, May 6-7, 2013, Revised Selected Papers 2. pp. 16–27. Springer (2013)

9. Mouha, N., Preneel, B.: Constraint-based differential cryptanalysis of block ciphers. In: International Symposium on Research in Attacks, Intrusions, and Defenses. pp. 139–158. Springer (2012)
10. Project, G.: Glpk (gnu linear programming kit). <https://www.gnu.org/software/glpk/> (2024)
11. of Standards, N.I., Technology: Advanced encryption standard (aes) (2001), <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>

Stage	Binary Index	Characteristic
Key	KL	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	KR	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
1. round input	R1	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	R0	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
1. round middle values	R2	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R3-3a-3b-3c-3d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
2. round input	R4	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R3	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
2. round middle values	R4a	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	R4b-4c-4d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
3. round input	R5	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R4	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
3. round middle values	R5a-5b-5c-5d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
4. round input	R6	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R5	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
4. round middle values	R6a	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	R6b-6c-6d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
5. round input	R7	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R6	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
5. round middle values	R7a	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R7b-7c-7d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
6. round input	R8	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R7	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
6. round middle values	R8a	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	R8b-8c-8d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
7. round input	R9	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R8	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
7. round middle values	R9a-9b-9c-9d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
8. round input	R10	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R9	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
8. round middle values	R10a	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	R10b-10c-10d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
9. round input	R11	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R10	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}

Table 1. Best 8 round related-key differential characteristic of ITUbee algorithm